

EECS 4413 Team A Project

*Mark Joseph Alconcel, Anania
Yohannes Tsegaye, Anh Tu Le, Yulia
Obshansky*

*218591115, 219077486, 219676287,
219736230*

Professor Alvine Belle

Apex Auto

Document Change Control

Version	Date	Authors	Summary of Changes
1.0	05/20	All	Create initial template
1.1	05/27	Anania Yohannes Tsegaye	Updated meeting logs
1.2		Yulia Obshansky	
1.3	06/07	Anh Tu Le	Aggregate the use case diagrams and sequence diagrams from different members, prepare the initial test cases, and complete the introduction section of the report.
1.4	06/09	Mark Joseph Alconcel	Architecture write-up, package diagram, component diagram, use case diagram, sequence diagram
1.5	06/10	Anania Yohannes Tsegaye	Added Backlogs and Gannt Chart Diagrams
1.6	06/11	All	Final version. Review the project documents together before final submission.

Document Sign-Off

Name (Position)	Signature	Date
Anh Tu Le	Anh Tu Le	06/11/2026
Yulia Obshansky		06/11/2026
Anania Yohannes Tsegaye	Anania Yohannes Tsegaye	06/11/2026
Mark Joseph Alconcel	Mark Joseph Alconcel	06/11/2026

Contents

- 1 INTRODUCTION**
 - 1.1 Purpose
 - 1.2 Overview
 - 1.3 Resources - References
- 2 MAJOR DESIGN DECISIONS**
 - 2.1 Three-Tier Client-Server Architecture
 - 2.2 Model-View-Controller (MVC)
 - 2.3 Data Access Object (DAO)
 - 2.4 High Cohesion and Low Coupling
 - 2.5 Single Page Application Front-End
 - 2.6 Relational Database Schema
- 3 USE CASE DIAGRAM**
- 4 SEQUENCE DIAGRAMS**
- 5 ARCHITECTURE**
- 6 ACTIVITIES PLAN**
 - 6.1 Project Backlog and Sprint Backlog
 - 6.2 Group Meeting Logs
- 7 TEST DRIVEN DEVELOPMENT**

1 Introduction

1.1 Purpose

The purpose of this project is to design an online electric vehicle marketplace called Apex Auto. The system will allow customers to browse available electric vehicles, view vehicle details, filter and sort listings, save favorite vehicles, add vehicles to a shopping cart, submit reviews, and receive support through a chatbot. It will also support user authentication and administrative functions for managing the system.

This document presents the software design for Apex Auto, including the major design decisions, use case diagram, sequence diagrams, system architecture, activity plan, and initial test cases.

1.2 Overview

Apex Auto is a web-based electric vehicle marketplace designed to help customers browse, compare, and purchase new or used electric vehicles online. The system solves the problem of providing a centralized digital store where customers can view vehicle details, filter and sort listings, save favorite vehicles, add vehicles to a shopping cart, submit reviews, and receive basic navigation support through a chatbot. Apex Auto is designed as a multi-tier web application with Vite, a React based frontend framework, Spring Boot back-end, REST APIs, and PostgreSQL, and a cloud-hosted relational database.

1.3 Resources - References

<https://plantuml.com/use-case-diagram>

<https://plantuml.com/sequence-diagram>

<https://www.geeksforgeeks.org/system-design/data-access-object-pattern/>

<https://docs.google.com/spreadsheets/u/2/?ftv=1&tgif=c>(Gantt Chart)

2 Major Design Decisions

2.1 Three-Tier Client-Server Architecture

The marketplace splits into three tiers: Presentation, Business Logic, and Data. The front-end runs as a React Single Page Application on Vercel or Netlify. The back-end runs on Spring Boot, deployed on Railway or Render, exposing a REST API over JSON with HTTPS in production. The database tier runs on a cloud-hosted PostgreSQL instance on Supabase or PlanetScale. By keeping each tier independent, we ensure the system can be developed, tested, and deployed in parallel without one layer blocking another.

2.2 Model-View-Controller (MVC)

Within the back-end, we follow the MVC pattern. Controllers like VehicleController, OrderController, AuthController, CartController, and AnalyticsController receive incoming HTTP requests and delegate work to the Service layer. The Service layer holds all business logic, keeping controllers focused solely on routing. Below that, the DAO/Repository layer handles all database queries through JDBC/JPA. Each layer only communicates with the one directly beneath it, which gives us high cohesion within layers and low coupling across them.

2.3 Data Access Object (DAO)

We centralize all database access in the DAO/Repository layer. Each domain entity, including Vehicle, Order, User, CartItem, and VisitEvent, gets its own dedicated DAO class responsible for its CRUD operations. This isolates our business logic from the SQL schema

entirely. When the schema changes, we update one place. When we test a service, we mock its DAO and move forward without touching the database.

2.4 High Cohesion and Low Coupling

We decompose the system into five feature-aligned modules: Catalog, Ordering, Authentication, Shopping Cart, and Analytics. Each module owns a clearly scoped set of responsibilities and interacts with other modules only through well-defined interfaces and public methods rather than reaching into each other's data directly. For example, the Catalog module exposes read-only vehicle queries that the Cart and Order modules consume, but the Cart module has no knowledge of how that data is stored. This keeps modules independent, prevents failures from spreading, and lets team members build and test in parallel.

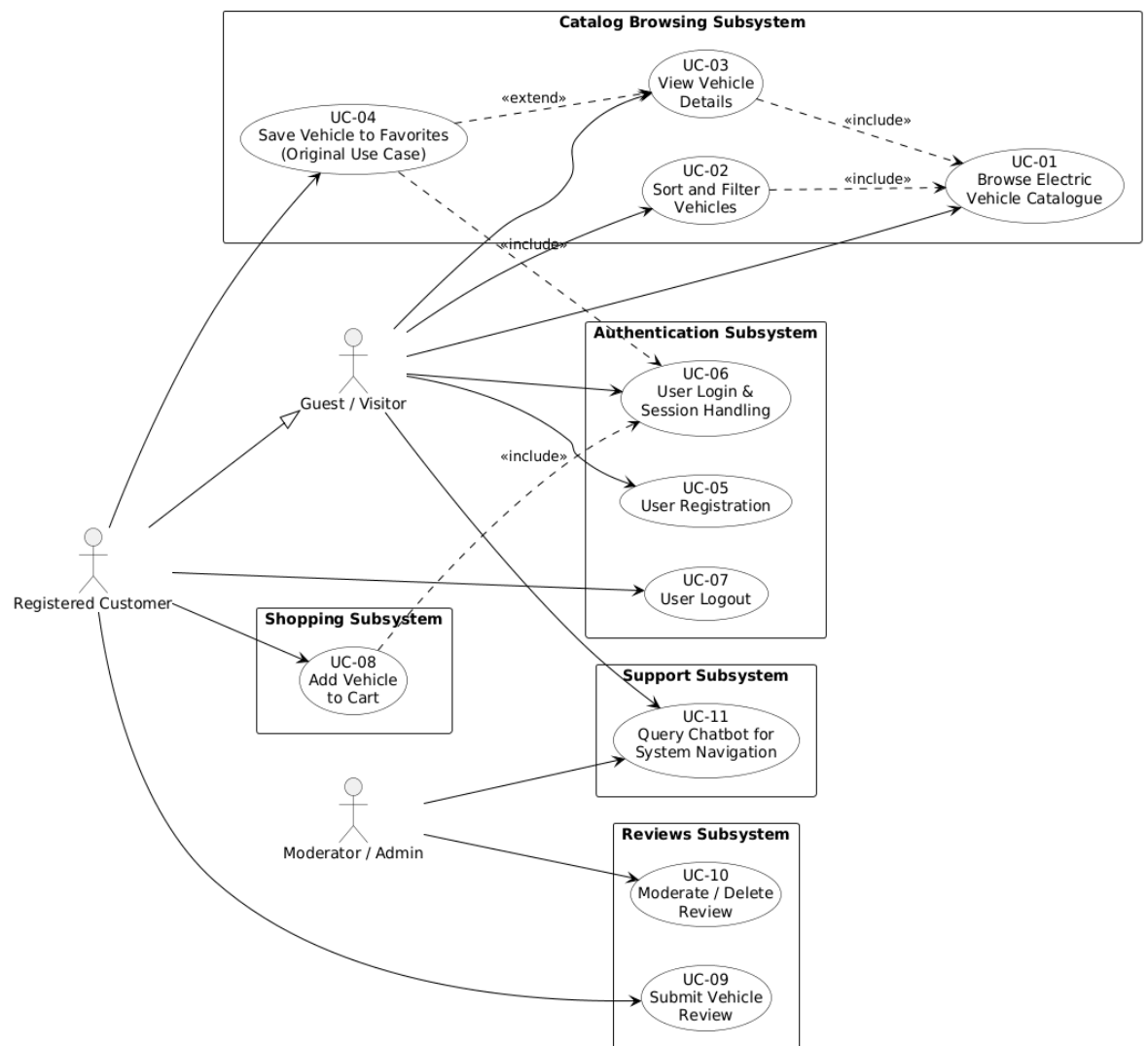
2.5 Single Page Application Front-End

We structure the front-end as a React SPA with five main views: Catalog for browsing, filtering, sorting, and comparing vehicles; Cart and Checkout for managing cart items, processing payments, and confirming orders; Authentication for registration, login, and logout; Admin for sales and usage reports; and Extras for the loan calculator and chatbot. The SPA communicates exclusively through the REST API, with no direct database access from the client. This enforces a clean boundary between presentation and business logic while keeping all production traffic under HTTPS.

2.6 Relational Database Schema

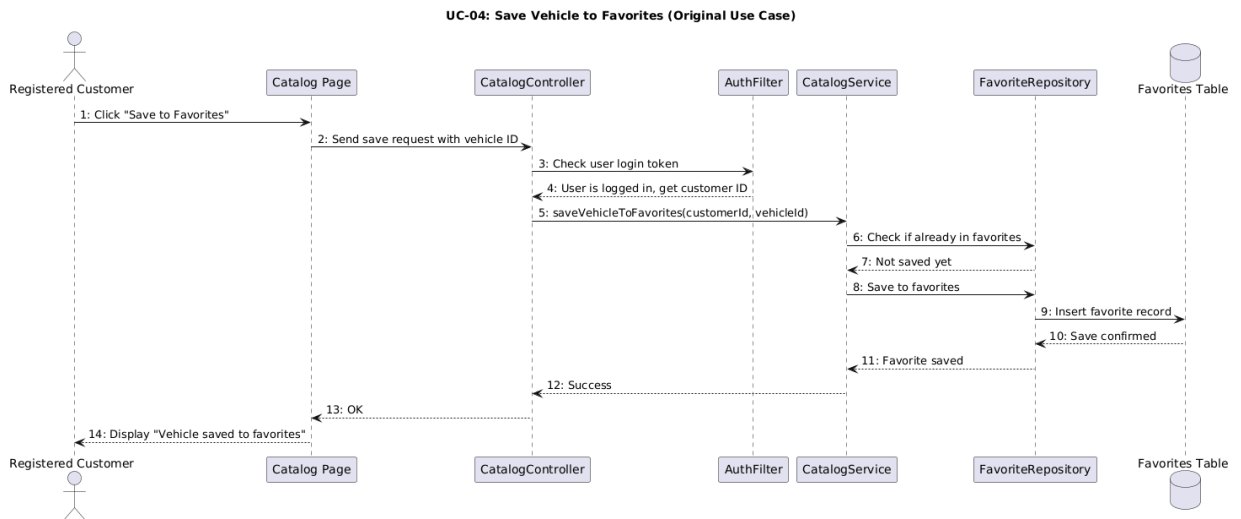
We use a cloud-hosted PostgreSQL database, extending the sample schema from the project specification to include tables for users, reviews, cart items, and visit events, as well as extended vehicle attributes like exterior colour, interior colour, interior fabric, and vehicle history reports. We chose a relational model over NoSQL because our core entities, including vehicles, orders, users, and reviews, have stable and well-defined relationships that benefit from referential integrity and transaction support. All schema access flows exclusively through the DAO layer, blocking SQL injection and enforcing consistent data access patterns across the application.

3 Use case Diagram



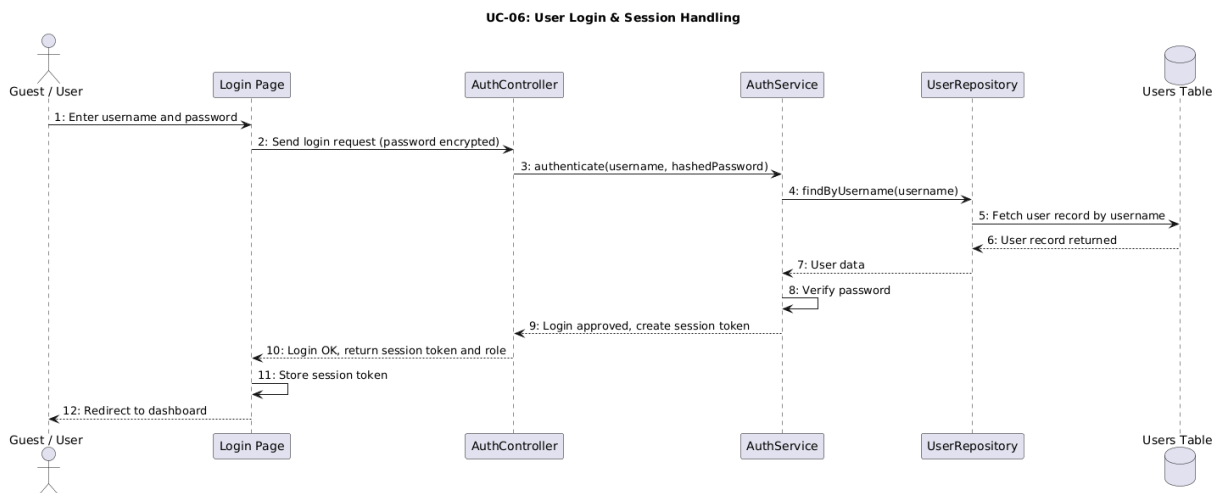
4 Sequence Diagrams

Sequence diagram for UC-04: Save Vehicle to Favorites (Original Use Case)



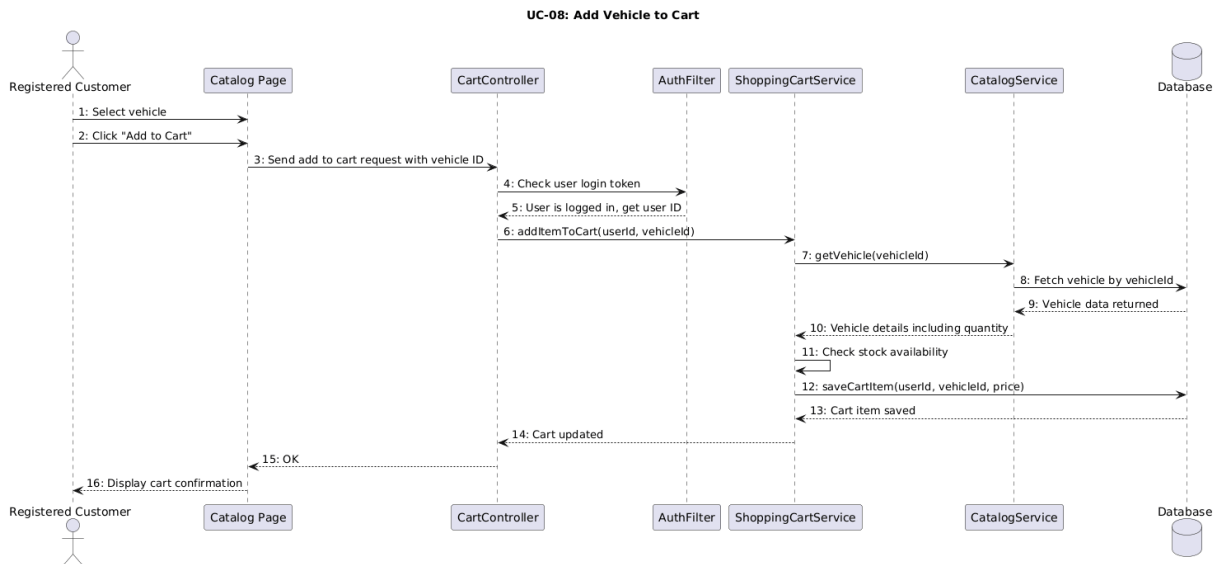
This sequence diagram shows how a registered customer saves a selected vehicle to their favorites list. The system checks the user's login token, confirms the customer identity, verifies that the vehicle has not already been saved, and then stores the favorite record in the database. A confirmation message is returned to the customer after the vehicle is successfully saved.

Sequence diagram for UC-06: User Login & Session Handling



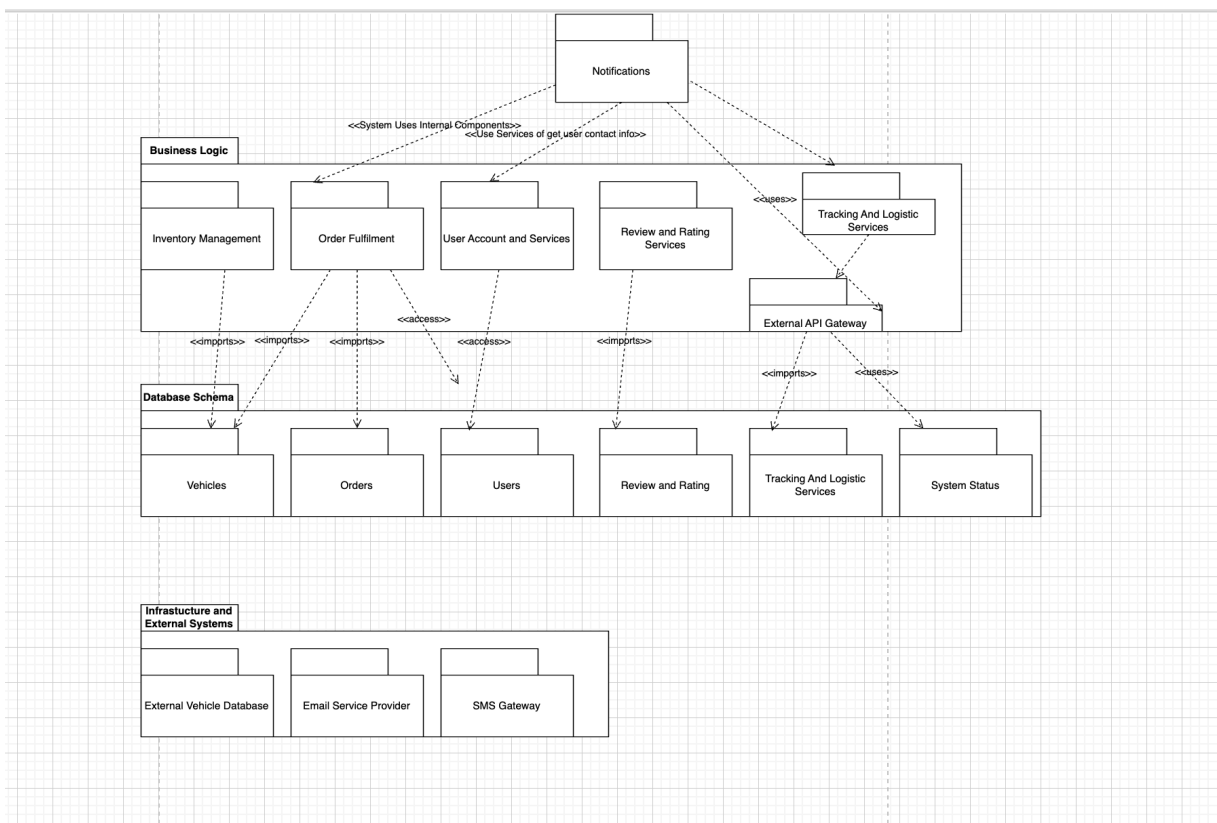
This sequence diagram shows how a guest or user logs into the system. The user enters login credentials, which are sent to the authentication controller and service. The service retrieves the user record, verifies the password, creates a session token, and returns the login result to the user interface. After successful login, the user is redirected to the dashboard or home page.

Sequence diagram for UC-08: Add Vehicle to Cart



This sequence diagram shows how a registered customer adds a vehicle to the shopping cart. The system checks the user's login token, retrieves the selected vehicle information, verifies stock availability, and saves the vehicle as a cart item. Once the cart is updated, the system displays a confirmation message to the customer.

5 Architecture



Modules			
Module Name	Description	Exposed Interface Names	Interface Description
M1: VehicleInventory	Manages vehicle listings, details, types, and history tracking.	M1:I1 (VehicleAccess) M1:I2 (VehicleManagement)	M1:I1: Read-only operations for searching and viewing vehicles. M1:I2: Administrative operations to add, update, or remove listings.
M2: OrderProcessing	Handles transactions, orders, and payment generation.	M2:I1 (CheckoutService)	M2:I1: Processes customer orders and initiates billing workflows.
M3: UserManagement	Manages user accounts, authentication tokens, and user profiles.	M3:I1 (AuthService) M3:I2 (UserProfileService)	M3:I1: Handles user login, registration, and security sessions. M3:I2: Manages account updates and user history.
M4: FeedbackService	Handles the collection and display of vehicle reviews and ratings.	M4:I1 (ReviewService)	M4:I1: Allows users to post ratings and pull review metrics for listings.

Interfaces		
Interface Name	Operations	Operation Descriptions
I1: VehicleAccess	List<Vehicle> getAllVehicles() (used by M2, M4) Vehicle getVehicleById(String vin) (used by M2, M4)	M1:I1:getAllVehicles(): Retrieves all active vehicle listings for marketplace browsing. M1:I1:getVehicleById(vin): Fetches granular details of a specific vehicle using its unique ID/VIN.

I2: VehicleManagement	boolean addVehicle(Vehicle v) (used by M3) boolean updateStatus(String vin, String status) (used by M2)	M1:I2:addVehicle(v): Allows a registered seller to publish a new vehicle to the marketplace. M1:I2:updateStatus(vin, status): Marks a vehicle's listing status (e.g., Sold, Pending, Available).
I3: CheckoutService	Order createOrder(User u, Vehicle v) (used by M3) boolean processPayment(Order o) (used by M3)	M2:I1:createOrder(u, v): Reserves a vehicle and builds an invoice structure for a buying user. M2:I1:processPayment(o): Interfaces with the payment processor to clear the transaction securely.
I4: AuthService	Token login(String email, String password) (used by M2, M4) boolean validateSession(Token t) (used by M2, M4)	M3:I1:login(...): Authenticates platform credentials and returns a session token. M3:I1:validateSession(t): Guards other routes by verifying if an incoming transaction request is authorized.
I5: ReviewService	boolean submitReview(Review r) (used by M3) List<Review> getVehicleReviews(String vin) (used by M1)	M4:I1:submitReview(r): Allows an authenticated user to write and save a rating after a purchase. M4:I1:getVehicleReviews(vin): Returns consumer feedback tied to a specific vehicle to append to its view page.

To implement our vehicle marketplace application, we will adopt a Service-Based Architecture, decomposing our system into highly cohesive, modular domain services that encapsulate the business logic and cleanly separate it from our relational database layout. Each primary business domain such as User, Vehicle, Orders, and Review will be managed by its own dedicated service class such as UserService, VehicleService, OrderService, and

ReviewService. These services will act as the sole orchestrators of their respective domains, executing business rules, managing transactions, and interacting with the database layer via data access objects (DAO). For instance, when a customer buys a car, the OrderService will handle the transaction creation and calculate totals, while internally utilizing the VehicleService to securely transition the vehicle's status to "Sold." By ensuring that modules only interact through explicitly defined interfaces and public methods rather than querying each other's underlying data structures directly, we achieve loose coupling, a highly scannable codebase, and a system that is easily maintainable and ready for future extension.

6 Activities Plan

1.1 Project Backlog and Sprint Backlog([Link for reference](#))

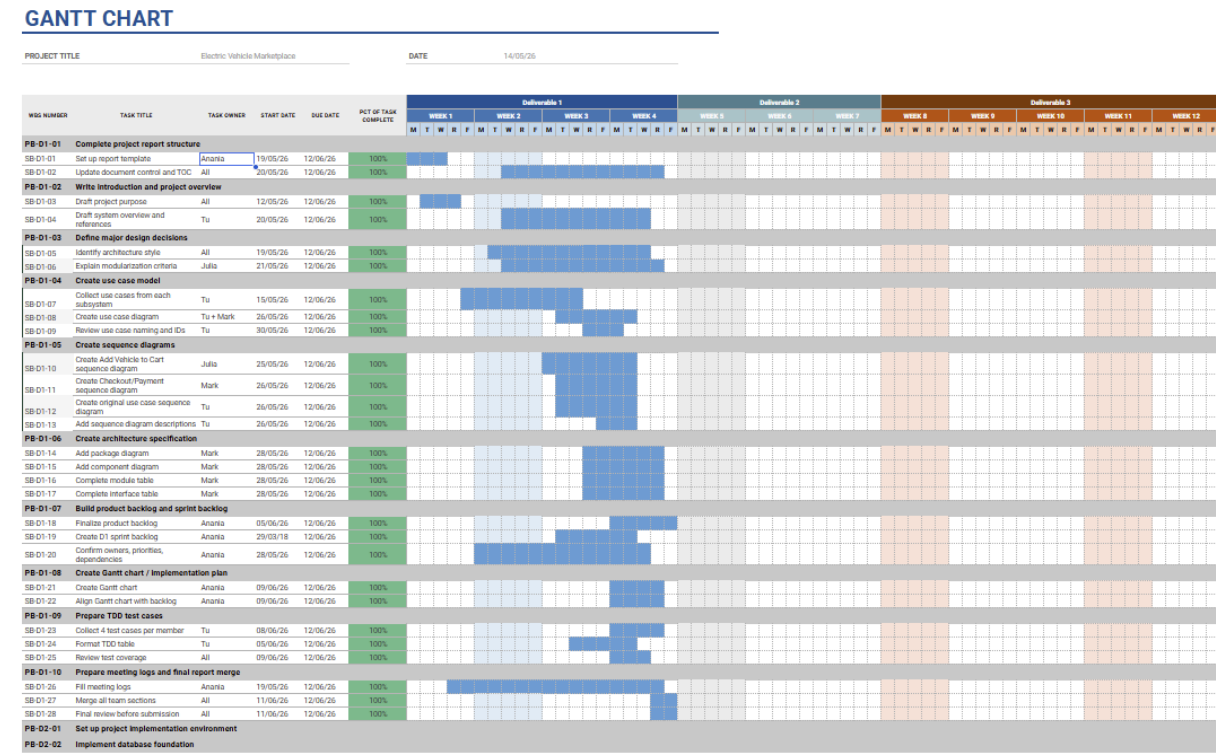
Product backlog:

Backlog ID	Backlog Item	Description	Owner	Related Use Case / Requirement	Priority	Target Deliverable	Dependencies	Status
PB-01-01	Complete project report structure	Set up the SDD template with title page, change control, table of contents, and section ownership.	All	Deliverable 1 report template	High	D1	Team member names and project title	Done
PB-01-02	Write introduction and project overview	Explain the purpose of the EV e-commerce system and summarize customer/admin features.	Yulia	Introduction section	High	D1	Project specification review	Done
PB-01-03	Define major design decisions	Describe architecture style, modularization, high cohesion, low coupling, and service-based design.	Mark	Major Design Decisions	High	D1	Architecture approach agreed by team	Done
PB-01-04	Create use case model	Build the use case diagram with customer, admin, and original use cases.	Tu	Use Case Diagram	High	D1	Final list of use cases	Done
PB-01-05	Create sequence diagrams	Create three sequence diagrams, including one original/distinguished use case.	All	Sequence Diagrams	High	D1	Final selected use cases	Done
PB-01-06	Create architecture specification	Add package/component diagrams and module/interface tables.	Mark	Architecture section	High	D1	Major services identified	Done
PB-01-07	Build product backlog and sprint backlog	Organize the project work by deliverable and create the D1 sprint backlog.	Anania	Activities Plan	High	D1	Team responsibilities confirmed	Done
PB-01-08	Create Gantt chart / implementation plan	Build the project schedule across D1, D2, and D3.	Anania	Gantt Chart	High	D1	Product backlog finalized	Done
PB-01-09	Prepare TDD test cases	Collect and format at least 16 test cases, 4 per member.	All	TDD section	High	D1	Use cases and feature ownership confirmed	Done
PB-01-10	Prepare meeting logs and final report merge	Record meeting minutes and combine all team sections into one report.	All	Group Meeting Logs / Final Report	High	D1	Teammates submit sections	Done
PB-02-01	Set up project implementation environment	Create shared repository, frontend/backend structure, and local development setup.	TBD	Implementation setup	High	D2	D1 design decisions completed	Planned
PB-02-02	Implement database foundation	Create database schema for users, vehicles, orders, reviews, and VisitEvent data.	TBD	Data Access / Database	High	D2	Final schema agreed	Planned
PB-02-03	Implement catalog browsing module	Allow users to view available electric vehicles.	TBD	UC-M1 / Catalog View	High	D2	Vehicle database exists	Planned
PB-02-04	Implement vehicle sorting and filtering	Allow users to sort/filter EVs by price, mileage, brand, shape, year, and history.	TBD	UC-M1 / Catalog filtering	High	D2	Catalog browsing module	Planned
PB-02-05	Implement vehicle details and comparison	Show full vehicle details/history and allow vehicle comparison.	TBD	UC-M2 / Compare vehicles	High	D2	Catalog data and catalog page	Planned
PB-02-06	Implement authentication and session handling	Add registration, login, logout, and protected session logic.	TBD	UC-R1 / UC-R2	High	D2	User database table	Planned
PB-02-07	Implement shopping cart module	Add, view, edit, and remove vehicles from the cart.	TBD	UC-C1 / UC-C2 / UC-M5	High	D2	Catalog and session handling	Planned
PB-02-08	Implement checkout and order processing prototype	Collect shipping/billing information and create order records.	TBD	UC-C3 / UC-P3 / UC-P4	High	D2	Cart module and user/session handling	Planned
PB-02-09	Implement payment simulation prototype	Approve two payments and deny every third request, updating order status.	TBD	Payment simulation requirement	High	D2	Order processing prototype	Planned
PB-02-10	Update report and sprint evidence for D2	Update report, screenshots, API evidence, backlog progress, and revised architecture.	TBD	Deliverable 2 report update	High	D2	D2 implementation progress	Planned
PB-03-01	Complete remaining customer features	Finish test details, loan calculator, customization, comparison, and UI polish.	TBD	Customer features	High	D3	Core D2 features implemented	Planned
PB-03-02	Complete reviews and ratings module	Allow users to submit and view vehicle reviews and five-star ratings.	TBD	Reviews and ratings requirement	High	D3	Auth and vehicle detail pages	Planned
PB-03-03	Complete chatbot/original feature	Implement chatbot or original use case and connect it to navigation/support.	TBD	Chatbot / Distinguished use case	High	D3	Frontend routes and catalog data	Planned
PB-03-04	Complete administrator analytics module	Build admin page for monthly sales and website usage reports.	TBD	UC-A1 / UC-A2	High	D3	Orders and VisitEvent data	Planned
PB-03-05	Complete security requirements	Protect checkout/confirm order, avoid plaintext passwords, and avoid credit card storage.	TBD	Security requirements	High	D3	Auth and checkout implemented	Planned
PB-03-06	Complete security testing	Run SQL injection and XSS test cases and document results.	TBD	SQL Injection / XSS tests	High	D3	Input forms and APIs implemented	Planned
PB-03-07	Complete performance testing	Test one backend API with multiple clients and record throughput/response time.	TBD	Performance requirement	High	D3	Backend API available	Planned
PB-03-08	Complete backend API and curl testing	Prepare curl scripts for backend API test cases.	TBD	Curl backend testing	High	D3	APIs implemented	Planned
PB-03-09	Deploy final application and database	Deploy web app and database to cloud/public address.	TBD	Cloud deployment requirement	High	D3	Final app and database ready	Planned
PB-03-10	Final report, source package, and demo preparation	Prepare final report, deployed link, GitHub/WAR/source files, SQL file, README, and demo flow.	TBD	Final submission package	High	D3	Final testing and deployment complete	Planned

Sprint Backlog:

Sprint Task ID	Related PB ID	Sprint Task	Output	Owner	Priority	Status	Due Date
SB-D1-01	PB-D1-01	Set up report template	SDD template prepared with title page and sections	Anania	High	In Progress	June 12
SB-D1-02	PB-D1-01	Update document control and TOC	Change control table and table of contents updated	All	High	In Progress	June 12
SB-D1-03	PB-D1-02	Draft project purpose	Purpose subsection for introduction	All	High	In Progress	June 12
SB-D1-04	PB-D1-02	Draft system overview and references	Overview and references subsection	Tu	High	In Progress	June 12
SB-D1-05	PB-D1-03	Identify architecture style	Multi-tier/service-based architecture notes	All	High	In Progress	June 12
SB-D1-06	PB-D1-03	Explain modularization criteria	High cohesion/low coupling explanation	Julia	High	In Progress	June 12
SB-D1-07	PB-D1-04	Collect use cases from each subsystem	Final use case list	Tu	High	In Progress	June 12
SB-D1-08	PB-D1-04	Create use case diagram	UML use case diagram with customer/admin/original use cases	Tu + Mark	High	In Progress	June 12
SB-D1-09	PB-D1-04	Review use case naming and IDs	Consistent UC IDs across report	Tu	High	In Progress	June 12
SB-D1-10	PB-D1-05	Create Add Vehicle to Cart sequence diagram	Sequence diagram for cart flow	Julia	High	In Progress	June 12
SB-D1-11	PB-D1-05	Create Authentication sequence diagram	Sequence diagram for checkout/payment	Mark	High	In Progress	June 12
SB-D1-12	PB-D1-05	Create original use case sequence diagram	Sequence diagram for original/distinguished feature	Tu	High	In Progress	June 12
SB-D1-13	PB-D1-05	Add sequence diagram descriptions	Short explanation under each diagram	Tu	High	In Progress	June 12
SB-D1-14	PB-D1-06	Add package diagram	Package diagram showing tiers/subsystems	Mark	High	In Progress	June 12
SB-D1-15	PB-D1-06	Add component diagram	Component diagram showing service interactions	Mark	High	In Progress	June 12
SB-D1-16	PB-D1-06	Complete module table	Three or more business modules described	Mark	High	In Progress	June 12
SB-D1-17	PB-D1-06	Complete interface table	Interface operations and descriptions completed	Mark	High	In Progress	June 12
SB-D1-18	PB-D1-07	Finalize product backlog	Product backlog classified by D1/D2/D3	Anania	High	In Progress	June 12
SB-D1-19	PB-D1-07	Create D1 sprint backlog	Sprint backlog tasks mapped to product backlog items	Anania	High	In Progress	June 12
SB-D1-20	PB-D1-07	Confirm owners, priorities, dependencies	Backlog table cleaned and finalized	Anania	High	In Progress	June 12
SB-D1-21	PB-D1-08	Create Gantt chart	Timeline across D1, D2, and D3	Anania	High	In Progress	June 12
SB-D1-22	PB-D1-08	Align Gantt chart with backlog	Dates and owners match backlog items	Anania	High	In Progress	June 12
SB-D1-23	PB-D1-09	Collect 4 test cases per member	16 total test cases collected	Tu	High	In Progress	June 12
SB-D1-24	PB-D1-09	Format TDD table	Test cases placed in required table format	Tu	High	In Progress	June 12
SB-D1-25	PB-D1-09	Review test coverage	Test cases checked against use cases/security requirements	All	High	In Progress	June 12
SB-D1-26	PB-D1-10	Fill meeting logs	Attendance, topics, decisions, and assigned tasks recorded	Anania	High	In Progress	June 12
SB-D1-27	PB-D1-10	Merge all team sections	One combined report document	All	High	In Progress	June 12
SB-D1-28	PB-D1-10	Final review before submission	Formatting, citations/references, diagrams, and consistency checker	All	High	In Progress	June 12

Gantt Chart:



1.2 Group Meeting Logs

In this Section you write minutes of each meeting, listing the attendance, what the topics of discussion in the meeting were, any decisions that were made, and which team members were assigned which tasks. These minutes must be submitted with the project report in each deliverable and will provide input to be used for the overall assessment of the project.

Present Group Members	Meeting Date	Issues Discussed / Resolved
Anania Tsegaye, Mark Joseph Alconcel, Anh Tu Le, Yulia Obshansky	2026-05-20	Introduction meeting. Members introduced themselves, reviewed the EV e-commerce project requirements, discussed the main expectations for the deliverables, and agreed to begin dividing responsibilities by system components.
Anania Tsegaye, Mark Joseph Alconcel, Anh Tu Le, Yulia Obshansky	2026-05-27	Discussed Deliverable 1 sections and confirmed the main system areas: catalog/browsing, cart/checkout/payment, authentication/reviews/chatbot, and analytics/testing/project management. Initial task ownership was assigned.
Anania Tsegaye, Mark Joseph Alconcel, Anh Tu Le, Yulia Obshansky	2026-06-3	Reviewed progress on the use case diagram, sequence diagrams, architecture section, backlog planning, and test cases. Members clarified dependencies between sections and identified missing information needed for the report.

Anania Tsegaye, Mark Joseph Alconcel, Anh Tu Le, Yulia Obshansky	2026-06-10	Team alignment meeting before submission. Members checked that major Deliverable 1 sections were being completed, confirmed remaining tasks, reviewed consistency between diagrams/backlog/Gantt chart/test cases, and assigned final fixes.
Anania Tsegaye, Mark Joseph Alconcel, Anh Tu Le, Yulia Obshansky	2026-06-11	Final review and submission meeting. Team reviewed the completed report, checked formatting and section consistency, confirmed diagrams and test cases were included, updated meeting logs, and prepared the final submission package.

7 Test Driven Development

The following 16 test cases cover the 11 use cases included in the current use case diagram for this deliverable.

Test ID	TC-01
Category	Catalog Browsing
Requirements Coverage	UC-01: Browse Vehicle Catalogue
Initial Condition	System is running. User is on the home page.
Procedure	1. User opens the website. 2. User navigates to the vehicle catalogue page. 3. System loads and displays all available vehicles.
Expected Outcome	All available vehicles are displayed on the catalogue page.
Notes	Only vehicles with status Available should be shown.

Test ID	TC-02
Category	Catalog Browsing
Requirements Coverage	UC-02: Sort Vehicles by Price
Initial Condition	The system is running. Vehicles are loaded in the catalogue.
Procedure	1. The user opens the catalogue. 2. User selects Sort by Price: Low to High. 3. System reorders and displays the vehicle list.
Expected Outcome	Vehicles are displayed in ascending order by price.

Notes	Should also work for descending order and sort by mileage.
--------------	--

Test ID	TC-03
Category	Catalog Browsing
Requirements Coverage	UC-02: Filter Vehicles by Brand
Initial Condition	System is running. Vehicles are loaded in the catalogue.
Procedure	1. User opens the catalogue. 2. User selects a brand filter (e.g. Tesla). 3. System filters and displays matching vehicles.
Expected Outcome	Only vehicles matching the selected brand are shown.
Notes	Filter should also support shape, model year, and accident history.

Test ID	TC-04
Category	Catalog Browsing
Requirements Coverage	UC-03: View Vehicle Details
Initial Condition	The system is running. At least one vehicle exists in the catalogue.
Procedure	1. The user opens the catalogue. 2. The user clicks on a specific vehicle. 3. The system loads the vehicle detail page.
Expected Outcome	Full vehicle details are displayed including price, description, history, and reviews.
Notes	

Test ID	TC-05
Category	Authentication
Requirements Coverage	UC-05: Successful User Registration
Initial Condition	System is running. User does not have an existing account.

Procedure	1. User navigates to the registration page. 2. User fills in name, email, and password. 3. User submits the form. 4. System validates and creates the account.
Expected Outcome	New account is created and user is redirected to the home page.
Notes	Email must be unique. Password must be encrypted before storing.

Test ID	TC-06
Category	Authentication
Requirements Coverage	UC-05: Registration with Duplicate Email
Initial Condition	System is running. An account with the email already exists.
Procedure	1. User navigates to the registration page. 2. User enters an email that is already registered. 3. User submits the form.
Expected Outcome	System rejects the registration and shows an error: Email already in use.
Notes	System must not create duplicate accounts.

Test ID	TC-07
Category	Authentication
Requirements Coverage	UC-06: Successful User Login
Initial Condition	System is running. User has a registered account.
Procedure	1. User navigates to the login page. 2. User enters valid email and password. 3. User clicks Login.
Expected Outcome	User is logged in and redirected to the home page with a valid session token.
Notes	Password must be verified against the encrypted stored version.

Test ID	TC-08
Category	Authentication
Requirements Coverage	UC-06: Login with Wrong Password
Initial Condition	System is running. User has a registered account.
Procedure	1. User navigates to the login page. 2. User enters correct email but wrong password. 3. User clicks Login.
Expected Outcome	System rejects login and shows: Invalid email or password.
Notes	The system should not reveal which field is incorrect for security reasons.

Test ID	TC-09
Category	Authentication
Requirements Coverage	UC-07: User Logout
Initial Condition	The system is running. The user is logged in.
Procedure	1. The user clicks the Logout button. 2. The system clears the session token. 3. The user is redirected to the home page.
Expected Outcome	The user is logged out. Protected pages are no longer accessible.
Notes	Session token must be fully cleared on logout.

Test ID	TC-10
Category	Shopping
Requirements Coverage	UC-08: Successfully Add Vehicle to Cart
Initial Condition	The system is running. The user is logged in. The vehicle is available in the catalogue.

Procedure	1. The user opens a vehicle detail page. 2. User clicks Add to Cart. 3. The system checks stock availability. 4. The system saves the vehicle to the user's cart.
Expected Outcome	The vehicle is added to the cart and a confirmation message is shown.
Notes	Users must be logged in. The system must verify vehicle availability before adding.

Test ID	TC-11
Category	Shopping
Requirements Coverage	UC-08: Add to Cart Without Login
Initial Condition	The system is running. The user is not logged in.
Procedure	1. The user opens a vehicle detail page. 2. User clicks Add to Cart.
Expected Outcome	The system redirects the user to the login page.
Notes	Cart actions must require authentication.

Test ID	TC-12
Category	Favorites
Requirements Coverage	UC-04: Save Vehicle to Favorites
Initial Condition	The system is running. The user is logged in. The vehicle exists in the catalogue.
Procedure	1. The user opens a vehicle detail page. 2. User clicks Save to Favorites. 3. The system checks if the vehicle is already saved. 4. The system saves the vehicle to the user's favorites.
Expected Outcome	The vehicle is added to the user's favorites list and a confirmation is shown.
Notes	Only registered and logged-in users can save favorites.

Test ID	TC-13
Category	Favorites
Requirements Coverage	UC-04: Save Already Favorited Vehicle
Initial Condition	The system is running. The user is logged in. The vehicle is already one of my favorites.
Procedure	1. Users open a vehicle detail page already in their favorites. 2. The user clicks Save to Favorites again.
Expected Outcome	System shows: Vehicle already in favorites. No duplicate is created.
Notes	Duplicate favorites must be prevented at the database level.

Test ID	TC-14
Category	Reviews
Requirements Coverage	UC-09: Submit a Vehicle Review
Initial Condition	The system is running. The user is logged in. The vehicle exists in the catalogue.
Procedure	1. The user navigates to a vehicle detail page. 2. User scrolls to the review section. 3. The user enters a star rating and written review. 4. The user submits the review.
Expected Outcome	Review is saved and displayed on the vehicle page with the user's name and rating.
Notes	Only authenticated users can submit reviews.

Test ID	TC-15
Category	Reviews
Requirements Coverage	UC-10: Admin Deletes a Review
Initial Condition	The system is running. Admin is logged in. A review exists on a vehicle.

Procedure	1. Admin navigates to the review management section. 2. Admin selects a review to delete. 3. Admin confirms the deletion.
Expected Outcome	Review is removed from the system and no longer appears on the vehicle page.
Notes	Only admins or moderators can delete reviews.

Test ID	TC-16
Category	Chatbot
Requirements Coverage	UC-11: Query Chatbot for Navigation
Initial Condition	System is running. User is on any page.
Procedure	1. User clicks the chatbot icon. 2. User types a question such as: Show me Tesla vehicles. 3. Chatbot processes the query and returns a response.
Expected Outcome	Chatbot responds with relevant links or results to help the user navigate.
Notes	Chatbot must be accessible to both guests and registered users.